

Date:
Exp No:2

Aim:implementation of Process and thread (Life cycle of process) :

i)Process creation and termination

ii)Thread creation and termination

i)Process creation and termination

A process goes through the following states:

1. **New** – Process is being created
2. **Ready** – Process is waiting to be assigned CPU
3. **Running** – Process is executing
4. **Waiting / Blocked** – Waiting for I/O or event
5. **Terminated** – Process has finished execution

- **Process creation** happens using `fork()`
- **Process termination** happens using `exit()`

1. Write the c program OS_exp2

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Process creation
    pid = fork();

    if (pid < 0) {
        printf("Process creation failed\n");
        exit(1);
    }

    // Child process
    if (pid == 0) {
        printf("\nChild Process Running\n");
        printf("Child PID: %d\n", getpid());
        printf("Child Parent PID: %d\n", getppid());

        printf("Child Process Terminated\n");
        exit(0); // Child termination
    }
}
```

```

}

// Parent process
else {
    wait(NULL); // Parent waits for child
    printf("\nParent Process Resumed\n");
    printf("Parent Process Terminated\n");
    exit(0);    // Parent termination
}
}

```

2.Compile and Run

```

gcc OS_exp2.c -o OS_exp2
./OS_exp2

```

3.Expected Output

```

Parent Process Started
Parent PID: 9117

Child Process Running
Child PID: 9118
Child Parent PID: 9117
Child Process Terminated

Parent Process Resumed
Parent Process Terminated

```

ii)Thread creation and termination

Life Cycle of a Thread (Short Theory)

A thread passes through the following states:

1. **New** – Thread is created
 2. **Runnable / Ready** – Waiting for CPU
 3. **Running** – Thread is executing
 4. **Blocked / Waiting** – Waiting for resource
 5. **Terminated** – Thread finishes execution
- **Thread creation** → `pthread_create()`
 - **Thread termination** → `pthread_exit()`

1. Write the c program OS_exp2.1

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Thread function
void* thread_function(void* arg) {
    printf("Thread Created and Running\n");
    printf("Thread ID: %lu\n", pthread_self());

    sleep(2); // Simulate work

    printf("Thread Terminated\n");
    pthread_exit(NULL); // Thread termination
}

int main() {
    pthread_t tid;

    printf("Main Thread Started\n");

    // Thread creation
    pthread_create(&tid, NULL, thread_function, NULL);

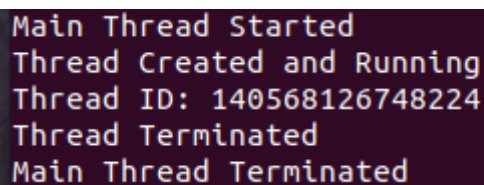
    // Wait for thread to finish
    pthread_join(tid, NULL);

    printf("Main Thread Terminated\n");
    return 0;
}
```

2.Compile and Run

```
gcc OS_exp2.1.c -o OS_exp2.1
./OS_exp2.1
```

3.Expected Output



```
Main Thread Started
Thread Created and Running
Thread ID: 140568126748224
Thread Terminated
Main Thread Terminated
```

4.Result:-the experiment on implementation of process and thread (life cycle of process) was successfully performed.